Hostel Management System

Branch Coverage Report

Branch (decision) coverage achieved by the automated test suite

Project	Hostel Management System (HMS)
Team	Team 3
Authors	Meghana P (PES2UG24CS642) B S Sharath Chandra Reddy (PES2UG24CS630) Adarsha V (PES2UG24CS621) Mohammadnoor Sirasgi (PES2UG25CS812)
Repository	github.com/adarshv15/hostel-management-system-team3 (latest tested working tree; commit not re-verified)
Baseline document	Software Requirements Specification v1.0 (TEAM_3_HMS_SRS)
Report version / date	1.1 / 10-11-2026
Status	Draft for team review
Test period	Latest local coverage run; 38-test suite

1. Introduction

Branch coverage checks whether the tests execute both outcomes of each decision in the application code (for example, the true and false paths of an if statement). A line can run while one of its outcomes remains untested, so branch coverage identifies gaps that statement coverage alone may miss. This report records the latest measured branch coverage for the Hostel Management System.

2. Measurement Method

Item	Detail
Operating system	Windows local development machine (exact build not recorded)
Python	Python 3.14.0
Django	Django 5.2.19 (previously recorded; local version not re-verified)
Coverage tool	coverage.py 7.16.2 with --branch
Database	SQLite, Django-managed test database
Command	python -m coverage run --branch --source=hostel,hms_project manage.py test
What a branch is	coverage.py records each possible jump from a decision line to the next line (or to the exit of the function or loop). One if normally gives two branches.
Scope	Application code in hostel/ and hms_project/. hostel/tests.py is excluded from the reported application coverage. Branches occur in models.py, admin.py and signals.py.
Execution date	10-11-2026

3. Summary of Results

Measure	Result
Branches in application code	26
Branches taken by the tests	26
Branches never taken	0
Decisions where only one outcome occurred (partial branches)	0
Branch coverage	100.0% (26 of 26)
Statement coverage (for comparison)	92.3% (299 of 324)
Combined figure printed by coverage.py	93% (coverage report Cover column; statements and branches combined)

Result: all 26 of the 26 measured application branches were taken, giving 100% branch coverage. The previous gaps in models.py, admin.py and signals.py are now covered. The signals module is loaded through the application configuration, and tests exercise both its audit-event creation and recursion-prevention paths. Statement coverage for comparison is 92.3% (299 of 324 executable statements).

3.1 Branch coverage by file

File	Branches	Taken	Missed	Partial	Coverage
hostel/models.py	20	20	0	0	100.0%
hostel/admin.py	2	2	0	0	100.0%
hostel/signals.py	4	4	0	0	100.0%
Total	26	26	0	0	100.0%

All other measured files (views.py, urls.py, settings.py, apps.py, the migration, asgi.py and wsgi.py) contain no branches in this coverage run.

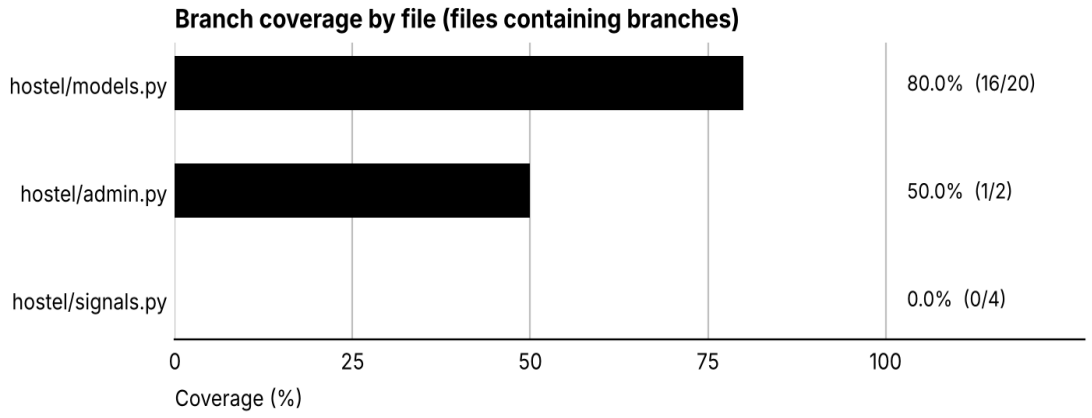


Figure 1. Branch coverage for each file that contains branches.

4. Branch-by-Branch Results

The table lists every measured application branch. The line locations refer to the current tested working tree; "exit" means the function or loop ends.

ID	Location	Decision	Outcome	Status
B-01	`models.py` 61	Active allocation for a student who is not ACTIVE?	Yes: raise ValidationError (61 to 62)	Covered
B-02	`models.py` 61	(same decision)	No: continue to capacity check (61 to 63)	Covered
B-03	`models.py` 63	Active allocation with a room?	Yes: check room capacity (63 to 64)	Covered
B-04	`models.py` 63	(same decision)	No: checked-out allocation or no room, skip check (63 to exit)	Covered
B-05	`models.py` 65	Allocation already saved (has pk)?	Yes: exclude itself from the count (65 to 66)	Covered
B-06	`models.py` 65	(same decision)	No: new allocation (65 to 67)	Covered
B-07	`models.py` 67	Active allocations at or above capacity?	Yes: raise "no available beds" (67 to 68)	Covered
B-08	`models.py` 67	(same decision)	No: allocation is valid (67 to exit)	Covered
B-09	`models.py` 98	Payment amount zero or less?	Yes: raise ValidationError (98 to 99)	Covered
B-10	`models.py` 98	(same decision)	No: continue (98 to 100)	Covered
B-11	`models.py` 100	Invoice and amount both present?	Yes: check against balance (100 to 101)	Covered
B-12	`models.py` 100	(same decision)	No: skip balance check (100 to exit)	Covered
B-13	`models.py` 102	Payment already saved (has pk)?	Yes: exclude itself from the paid total (102 to 103)	Covered
B-14	`models.py` 102	(same decision)	No: new payment (102 to 104)	Covered
B-15	`models.py` 109	Amount greater than balance due?	Yes: raise ValidationError (109 to 110)	Covered
B-16	`models.py` 109	(same decision)	No: payment is valid (109 to exit)	Covered
B-17	`models.py` 145	Leave end date earlier than start date?	Yes: raise ValidationError (145 to 146)	Covered
B-18	`models.py` 145	(same decision)	No: dates valid or missing (145 to exit)	Covered
B-19	`models.py` 162	Visitor exit time earlier than entry time?	Yes: raise ValidationError (162 to 163)	Covered
B-20	`models.py` 162	(same decision)	No: valid, or visit ongoing (162 to exit)	Covered
B-21	`admin.py` 81	Any audit events built for bulk delete?	Yes: bulk_create them (81 to 82)	Covered
B-22	`admin.py` 81	(same decision)	No: empty queryset, skip (81 to 84)	Covered
B-23	`signals.py` 41	Instance is an AuditEvent?	Yes: return, avoid recursion (41 to 42)	Covered
B-24	`signals.py` 41	(same decision)	No: create audit event (41 to 44)	Covered
B-25	`signals.py` 72	Another tracked model to register?	Yes: connect signals (72 to 73)	Covered
B-26	`signals.py` 72	(same decision)	No: loop ends (72 to exit)	Covered

26 of 26 branches covered; no missed or partial branches.

5. Analysis of Previously Missed Branches

ID	Location	What was previously not exercised	Latest test evidence / result
B-01	models.py 61	Allocation validation for a student who is not ACTIVE.	Covered by test_allocation_for_inactive_student_is_rejected; the invalid allocation is rejected.

ID	Location	What was previously not exercised	Latest test evidence / result
B-04	models.py 63	Allocation path where no room is assigned / room-capacity validation is skipped.	Covered by test_allocation_without_room_skips_room_capacity_validation.
B-12	models.py 100	Payment validation when no invoice is set.	Covered by test_payment_without_invoice_skips_balance_validation.
B-18	models.py 145	Leave request with valid dates passing clean().	Covered by test_valid_leave_dates_pass_model_validation.
B-22	admin.py 81	Bulk delete on an empty queryset.	Covered by test_admin_bulk_delete_with_empty_queryset.
B-23 to B-26	signals.py 41, 72	AuditEvent recursion guard and signal-registration loop outcomes.	Covered by test_audit_event_does_not_create_another_audit_event and signal registration tests; all four signal branches are taken.

The five previously missed application branch paths (B-01, B-04, B-12, B-18 and B-22) are now covered by additional tests. The four signals.py branches (B-23 to B-26) are also covered after signal registration and execution were added. No branch gaps remain in the measured application scope.

6. Limitations

- coverage.py measures jumps between lines, not each part of a compound condition. For example, line 61 combines multiple tests with and; branch coverage does not prove every individual condition was independently true and false. It is weaker than condition or MC/DC coverage.
- Branch coverage shows what ran, not whether the results were asserted correctly.
- Implicit branches inside Django (ORM, admin, forms) are not part of the measurement.
- The figures come from a local Windows run using Python 3.14.0 and coverage.py 7.16.2. The local Django version was not re-verified for this run. The project's CI environment was not measured.
- The Test Plan's branch-coverage target was not available in the repository. The measured branch coverage is 100% (26 of 26).

7. How to Reproduce

```
python -m coverage erase
python -m coverage run --branch --source=hostel,hms_project manage.py test
python -m coverage report -m --omit="*/tests.py"   # "N->M" and "N->exit" in Missing are branches
python -m coverage html --omit="*/tests.py"        # inspect detailed coverage in
htmlcov/index.html
```

8. Conclusion

The latest run covers all 26 measured application branches (100%). The previously uncovered allocation, payment, leave, bulk-delete and audit-signal paths are now exercised. Statement coverage is 92.3%; the remaining uncovered statements are reported separately in the Code Coverage Report. Full branch coverage is a positive result, but it does not replace assertions about correctness or broader system validation.

Sign-off

Role	Name	Signature	Date
Prepared by			
Reviewed by			
Approved by (Project Guide)			

Appendix A. Raw coverage.py Output

Output of coverage report -m --omit="*/tests.py" from the latest local run. The Cover column combines statement and branch coverage when --branch is enabled; the branch headline above is calculated from the 26 application branches only.

```
Name                                    Stmts   Miss Branch BrPart  Cover   Missing
-----------------------------------------------------------------------------------
hms_project\asgi.py                         4      4      0      0     0%   1-4
hms_project\settings.py                    25      0      0      0   100%
hms_project\urls.py                         4      0      0      0   100%
hms_project\wsgi.py                         4      4      0      0     0%   1-4
hostel\admin.py                            91      8      2      0    91%   45-58, 152-159, 169, 173-
174
hostel\apps.py                              7      0      0      0   100%
hostel\migrations\0001_initial.py           9      0      0      0   100%
hostel\models.py                          155      9     20      0    95%   71, 84, 115, 130, 149,
166, 180, 191, 206
hostel\signals.py                          17      0      4      0   100%
hostel\views.py                             8      0      0      0   100%
-----------------------------------------------------------------------------------
TOTAL                                     324     25     26      0    93%


3 empty files skipped.
```